

Hi Thanks For Commenting on my recent reel, Here's Detailed RAG Guide :

The Complete RAG Guide

What is RAG?

RAG stands for Retrieval-Augmented Generation.

It is an approach that allows an AI model to retrieve relevant information from an external knowledge source before generating an answer.

Instead of expecting the model to already know everything, RAG gives it the information it needs at the moment the question is asked.

The basic flow is:

Documents → Chunking → Embeddings → Vector Database → Retrieval → LLM → Answer

1. Why Do We Need RAG?

Imagine you have a **200-page company handbook** and want an AI assistant that can answer questions about it.

You could send the entire document to the model every time someone asks:

“What is our parental leave policy?”

But that creates several problems.

Problem 1: Context limits

AI models can only process a certain amount of information at once.

Large documents may exceed that limit.

Problem 2: Higher cost

Sending hundreds of pages with every request means processing large numbers of tokens repeatedly.

Problem 3: Irrelevant information

Most of the document has nothing to do with the user's specific question.

Problem 4: Slower responses

More input generally means more information for the system to process.

RAG solves this by finding the **small amount of relevant information first** and giving only that context to the model.

2. The Complete RAG Flow

A simple RAG pipeline looks like this:

Document

↓

Split into chunks

↓

Create embeddings

↓

Store embeddings

↓

User asks a question

↓

Embed the question

↓

Search for similar chunks

↓

Retrieve relevant context

↓

Send context + question to LLM

↓

Generate answer

There are really two phases:

Phase 1: Indexing

This happens before the user asks a question.

Load → Chunk → Embed → Store

Phase 2: Retrieval and Generation

This happens when the user asks something.

Question → Embed → Search → Retrieve → Generate

3. Step One: Load the Data

The first step is collecting the information you want the AI system to use.

This could include:

- PDFs
- Word documents
- Websites
- Notion pages
- Confluence pages
- Support articles
- Product documentation
- Database records
- Research papers
- Internal policies
- Meeting transcripts

The goal is to turn these sources into text that can be processed.

Example

Suppose your organization has:

- 100 HR documents
- 50 policy PDFs
- 500 support articles
- 1,000 internal wiki pages

A RAG system can create a searchable knowledge layer across all of them.

4. Step Two: Chunking

Once the text is extracted, the document is divided into smaller pieces called **chunks**.

Instead of treating a 200-page PDF as one giant block, you might split it into hundreds of smaller sections.

For example:

Original document

Employees are eligible for parental leave after completing six months of continuous employment. Full-time employees may receive up to 16 weeks of paid leave...

Could become:

Chunk 1

Eligibility requirements

Chunk 2

Length of parental leave

Chunk 3

Application process

5. Why Chunking Matters

Chunking has a major impact on RAG quality.

If chunks are too large:

- More irrelevant information gets retrieved.
- More tokens are passed to the model.
- Search becomes less precise.

If chunks are too small:

- Important context can be separated.
- Sentences may lose meaning.
- Retrieval may return incomplete information.

The goal is to create chunks that are **small enough for accurate retrieval but large enough to preserve meaning**.

6. Common Chunking Strategies

Fixed-Length Chunking

Split text after a certain number of characters or tokens.

Example:

500 tokens → next 500 tokens → next 500 tokens

Simple, but it may cut through important sections.

Overlapping Chunking

Chunks share some text with neighboring chunks.

Example:

Chunk 1: Tokens 1–500

Chunk 2: Tokens 450–950

Chunk 3: Tokens 900–1400

Overlap helps preserve context across boundaries.

Paragraph-Based Chunking

Each paragraph or group of paragraphs becomes a chunk.

Useful when documents are naturally well structured.

Section-Based Chunking

Split based on:

- Headings
- Chapters
- Sections
- Topics

This often preserves meaning better than arbitrary character counts.

Semantic Chunking

Split the document when the meaning or topic changes.

This can create more logically coherent chunks.

7. Step Three: Embeddings

After chunking, each chunk is converted into an **embedding**.

An embedding is a numerical representation of the meaning of text.

You can think of it as turning language into coordinates.

For example:

“How much parental leave do employees receive?”

and

“Employees receive 16 weeks of parental leave.”

would likely have similar embeddings because their meanings are related.

Even though the sentences do not use exactly the same words, their semantic meaning is close.

8. Why Embeddings Are Important

Traditional keyword search looks for matching words.

If you search:

“Vacation allowance”

A keyword system may struggle if the document uses:

“Annual paid leave”

Embeddings allow semantic search.

The system can understand that:

Vacation allowance $\approx$ Annual paid leave

This makes retrieval much more flexible.

9. Tokens vs Embeddings

These two concepts are often confused.

Tokens

Tokens are pieces of text a language model reads.

Example:

“Artificial intelligence is powerful.”

may be broken into several tokens.

Embeddings

Embeddings are vectors that represent semantic meaning.

So:

Tokens = units the model processes

Embeddings = numerical representations used for similarity

10. Step Four: Store the Embeddings

Once embeddings are created, they need to be stored somewhere.

This is typically done using a **vector database**.

Examples from the roadmap include:

- ChromaDB
- Pinecone
- Weaviate

Each stored record may contain:

Vector embedding

plus metadata such as:

- Original text
 - Document name
 - Page number
 - Section
 - Author
 - Date
 - URL
 - Category
-

11. What is a Vector Database?

A traditional SQL database is optimized for structured queries.

For example:

Find all customers where country = "India"

A vector database is optimized for similarity search.

For example:

Find the pieces of text most similar in meaning to “What is our refund policy?”

The vector database compares the user's question with stored vectors and returns the closest matches.

12. SQL Database vs Vector Database

SQL Database

Best for:

- Exact filters
- Structured data
- Relationships
- Transactions
- Aggregations

Example:

Find all orders above \$500.

Vector Database

Best for:

- Semantic search
- Similarity
- Recommendations
- Document retrieval
- RAG

Example:

Find documents discussing delayed shipments.

In real systems, SQL databases and vector databases can also be used together.

13. Step Five: User Asks a Question

Suppose the user asks:

“How many weeks of parental leave do employees receive?”

The system does not immediately send this question to the LLM.

First, it needs to retrieve relevant information.

14. Step Six: Embed the Question

The user's question is converted into an embedding using the same type of embedding model used for the document chunks.

Now both the question and the documents exist in the same mathematical representation space.

The system can compare them.

15. Step Seven: Similarity Search

The vector database compares the question embedding with stored document embeddings.

It might retrieve:

Result 1

Full-time employees may receive up to 16 weeks of paid parental leave.

Result 2

Employees become eligible after six months of continuous employment.

Result 3

Leave requests must be submitted to HR.

These chunks are considered most relevant to the question.

16. How Similarity is Measured

Vector systems typically use mathematical similarity measures.

Common approaches include:

- Cosine similarity
- Dot product
- Euclidean distance

Conceptually, the system asks:

“Which stored vectors are closest to the question vector?”

The closest vectors are likely to represent the most relevant information.

17. Step Eight: Retrieve the Best Context

The system might retrieve the top:

- 3 chunks
- 5 chunks
- 10 chunks

This is often called **Top-K retrieval**.

For example:

Top-K = 5

means:

Retrieve the five most relevant chunks.

More retrieved chunks do not automatically mean better answers.

Too much context can introduce noise.

18. Step Nine: Build the Prompt

The retrieved information is combined with the user's question.

The prompt might conceptually look like:

Instructions

Answer using only the provided context.

Context

Full-time employees may receive up to 16 weeks of paid parental leave.

Employees become eligible after six months of continuous employment.

Question

How many weeks of parental leave do employees receive?

19. Step Ten: Generate the Answer

The LLM now receives the question plus relevant information.

It can answer:

Full-time employees may receive up to 16 weeks of paid parental leave.

The model is generating the response, but the supporting knowledge comes from the retrieved documents.

That is why the approach is called:

Retrieval-Augmented Generation

Retrieval provides the knowledge.

Generation produces the answer.

20. RAG Architecture

A practical architecture looks like:

Data Sources

PDFs
Websites
Databases
Docs
Knowledge bases

↓

Document Loader

↓

Text Processing

↓

Chunking

↓

Embedding Model

↓

Vector Database

Then, during a query:

User

↓

Question

↓

Embedding Model



Vector Search



Relevant Chunks



Prompt



LLM



Answer

21. Simple RAG Example

Imagine an AI assistant for a university.

Its knowledge base contains:

- Admission policies
- Course information
- Scholarship rules
- Campus policies
- Exam guidelines

A student asks:

“What GPA do I need to maintain my scholarship?”

The RAG system:

1. Embeds the question



2. Searches the vector database

↓

3. Finds the scholarship-policy section

↓

4. Retrieves the relevant paragraph

↓

5. Sends it to the LLM

↓

6. Generates the answer

Without RAG, the model might rely on its training knowledge.

With RAG, the answer can be grounded in the university's own documents.

22. Retrieval Quality Determines RAG Quality

A common mistake is assuming the LLM is the most important part of a RAG application.

In reality:

Bad Retrieval → Bad Context → Bad Answer

Even a powerful model cannot reliably answer from information that was never retrieved.

That means improving RAG often involves improving:

- Document parsing
- Chunking
- Embeddings
- Metadata
- Retrieval
- Ranking
- Filtering

- Evaluation
-

23. Metadata Filtering

Suppose your knowledge base contains documents from:

- HR
- Finance
- Legal
- Engineering

A user asks:

“What is our annual leave policy?”

Instead of searching everything, you could filter:

Department = HR

Then perform vector search.

Metadata filters can include:

- Department
- Date
- Document type
- Customer
- Country
- Product
- Security level
- Author

This can improve retrieval accuracy.

24. Re-Ranking

Basic vector search may retrieve 20 possible chunks.

A second model or ranking system can then reorder them.

The flow becomes:

Question

↓

Retrieve 20 chunks

↓

Re-rank the 20

↓

Keep the best 5

↓

Send them to the LLM

This can improve context quality.

25. Hybrid Search

Vector search is excellent for semantic similarity.

Keyword search is excellent for exact terms.

Hybrid search combines both.

Example:

The user searches for:

“Error code AZ-1047”

Semantic search might not always be ideal for a very specific code.

Keyword search can match:

AZ-1047

exactly.

Hybrid search can combine:

Keyword Search + Vector Search

This is useful in technical documentation, legal documents, product catalogs, and enterprise search.

26. RAG and Citations

A strong RAG application can also return the source used to generate the answer.

Example:

Employees receive 16 weeks of paid parental leave.

Source: HR Handbook, Page 47

This gives users the ability to verify the answer.

Useful citation metadata includes:

- Document title
 - Page number
 - URL
 - Section
 - Date
 - Author
-

27. RAG vs Fine-Tuning

These two approaches solve different problems.

RAG

Adds external knowledge at query time.

Best when information:

- Changes frequently

- Comes from company documents
- Needs citations
- Must remain current
- Is too large to include permanently

Fine-Tuning

Further trains a model on examples.

Best when you want to change:

- Style
- Behavior
- Output structure
- Task performance
- Domain-specific response patterns

Simple memory trick:

RAG = Give the model knowledge

Fine-tuning = Change how the model behaves

They can also be used together.

28. RAG vs Long Context

Modern models can process large context windows, so why use RAG?

Because simply sending everything can still create problems:

- Higher token usage
- Higher cost
- More irrelevant information
- More difficult source management
- Slower processing
- Poorer control over retrieval

Long context and RAG are not necessarily competitors.

A system can retrieve relevant information first and then use a large context window when needed.

29. RAG Does Not Eliminate Hallucinations

RAG can help ground responses, but it does not guarantee perfect answers.

The model can still:

- Misinterpret retrieved information
- Combine sources incorrectly
- Ignore relevant context
- Produce unsupported claims

That is why production systems often include:

- Citations
 - Confidence checks
 - Retrieval evaluation
 - Output validation
 - Guardrails
 - Human review for sensitive use cases
-

30. Common RAG Failure Modes

Failure 1: Wrong Chunk Retrieved

The relevant answer exists, but retrieval returns something else.

Fix

Improve:

- Chunking
- Embeddings
- Metadata
- Hybrid search
- Re-ranking

Failure 2: Relevant Information Split Across Chunks

The answer requires information from two nearby sections.

Fix

Use:

- Chunk overlap
- Larger chunks
- Parent-child retrieval

Failure 3: Too Many Chunks

The prompt becomes overloaded with irrelevant information.

Fix

Reduce Top-K or improve ranking.

Failure 4: Outdated Documents

The vector database contains old versions.

Fix

Build document versioning and re-indexing workflows.

Failure 5: Missing Source Information

Users cannot verify the answer.

Fix

Store document metadata and generate citations.

31. Production RAG Pipeline

A production-grade RAG system may look like:

Data Sources



Ingestion Pipeline



Document Parsing



Cleaning



Chunking



Embedding Generation



Vector Database



Metadata Index



User Query



Query Understanding



Metadata Filtering



Hybrid Retrieval



Re-Ranking



Context Assembly



LLM



Answer Validation



Citation Generation



Monitoring & Feedback

32. Important Components of Production RAG

Data Ingestion

Automatically detect new or updated documents.

Re-Indexing

Update embeddings when information changes.

Access Control

Users should only retrieve documents they are authorized to see.

Monitoring

Track:

- Retrieval quality
- Response quality
- Latency
- Cost
- Failure rate

Feedback

Allow users to mark responses:

- Helpful
- Incorrect
- Missing information

That feedback can improve the system.

33. RAG Evaluation

You should evaluate both retrieval and generation.

Retrieval Evaluation

Ask:

- Did we retrieve the correct document?
- Did we retrieve the correct chunk?
- Was the answer present in the retrieved context?

Possible metrics include:

- Recall
- Precision

- Hit rate
- Mean Reciprocal Rank

Generation Evaluation

Ask:

- Is the answer correct?
 - Is it supported by the context?
 - Is it relevant?
 - Does it contain unsupported claims?
 - Are citations accurate?
-

34. RAG Tech Stack

A typical RAG stack may include:

Data Sources

PDF
Notion
Websites
Databases
Cloud storage

Embedding Layer

Embedding model

Vector Storage

ChromaDB
Pinecone
Weaviate

Application Layer

Python
APIs
Backend services

LLM Layer

A language model

Evaluation Layer

Custom tests

Human feedback

Automated evaluation

The exact technologies matter less than understanding the architecture.

35. Beginner RAG Project

Build a **PDF Question-Answering Assistant**.

Step 1

Upload a PDF.

Step 2

Extract the text.

Step 3

Split it into chunks.

Step 4

Create embeddings.

Step 5

Store them in a vector database.

Step 6

Create a question input.

Step 7

Embed the question.

Step 8

Retrieve the top relevant chunks.

Step 9

Send those chunks with the question to an LLM.

Step 10

Display the answer with source references.

This one project teaches almost the entire RAG workflow.

36. Intermediate RAG Project

Build an **Internal Company Knowledge Assistant**.

Connect:

- Policies
- Notion
- Google Drive
- Documentation
- FAQs

Add:

- Metadata filters
 - Citations
 - User authentication
 - Access control
 - Conversation history
 - Feedback
-

37. Advanced RAG Project

Build an **Enterprise Research Assistant**.

Capabilities:

- Search thousands of documents
 - Combine multiple sources
 - Hybrid retrieval
 - Re-ranking
 - Query rewriting
 - Metadata filters
 - Citations
 - User-level permissions
 - Evaluation
 - Monitoring
 - Audit logs
-

38. RAG Learning Roadmap

Stage 1: Foundations

Learn:

- Python
- APIs
- JSON
- LLM basics
- Tokens
- Prompt engineering

Stage 2: Embeddings

Understand:

- Embeddings
- Semantic similarity
- Vector search
- Distance metrics

Stage 3: Vector Databases

Practice:

- Creating collections
- Adding vectors
- Metadata storage
- Similarity search
- Filtering

Stage 4: Build Basic RAG

Implement:

Document → Chunk → Embed → Store → Retrieve → Generate

Stage 5: Improve Retrieval

Learn:

- Chunk overlap
- Metadata filtering
- Hybrid search
- Re-ranking
- Query rewriting

Stage 6: Productionize

Add:

- Security
 - Observability
 - Evaluation
 - Citations
 - Caching
 - Error handling
 - Access control
 - Monitoring
-

39. RAG in One Example

User asks:

“What is the company's remote-work policy?”

Without RAG:

Question → LLM → Answer based mainly on model knowledge

With RAG:

Question

↓

Create query embedding

↓

Search internal policy database

↓

Retrieve remote-work section

↓

Send section + question to LLM

↓

Generate grounded answer

↓

Show source

That is the core value of RAG.

40. The Most Important RAG Concepts to Remember

Chunking

Break large documents into smaller meaningful sections.

Embeddings

Convert text into numerical representations of meaning.

Vector Database

Store and search embeddings.

Retrieval

Find the most relevant information for a question.

Re-Ranking

Improve the order of retrieved results.

Context

Relevant information supplied to the LLM.

Generation

The LLM produces the final response.

Citations

Show users where the information came from.

Evaluation

Measure whether retrieval and answers are actually correct.

Quick RAG Cheat Sheet

Concept	Meaning
RAG	Retrieval-Augmented Generation
Chunk	Small section of a larger document
Embedding	Numerical representation of semantic meaning
Vector Database	Database optimized for vector similarity search
Retrieval	Finding relevant chunks
Top-K	Number of chunks retrieved
Metadata	Additional information about a chunk
Re-Ranking	Reordering results by relevance

Hybrid Search	Combining semantic and keyword search
Context	Retrieved information sent to the LLM
Generation	LLM creates the final answer
Citation	Reference showing where the answer came from

The One-Line Definition

RAG is a technique where an AI system retrieves relevant information from an external knowledge source and gives that information to an LLM before it generates an answer.

The easiest flow to remember is:

Chunk → Embed → Store → Retrieve → Generate

And the most important rule is:

A RAG system is only as good as the context it retrieves.

Follow me for more such insights

Instagram : www.instagram.com/chandukongara_

Linkedin : <https://www.linkedin.com/in/poornachandra-kongara/>

Substack : <https://substack.com/@chanduyourdataguy>

Youtube : https://www.youtube.com/@worldofchandus?sub_confirmation=1